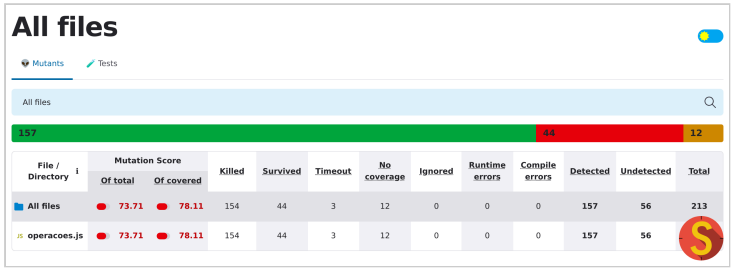
Análise de Eficácia de Testes com Teste de Mutação

Henrique Lima Volponi

1. Análise inicial

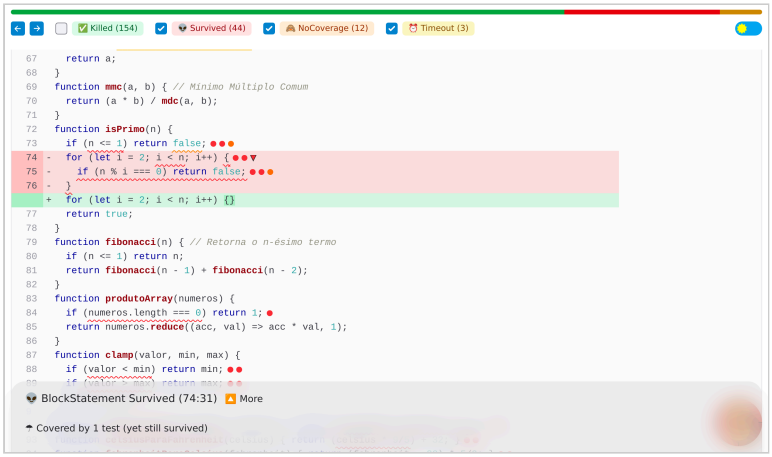
Métrica	Valor inicial
Cobertura de statements	85,41%
Cobertura de branches	58,82%
Cobertura de funções	100%
Cobertura de linhas	98,64%
Mutation score (Stryker)	**73,71%**



A suíte original executava 98,64% das linhas, mas o Stryker mostrou que 56 dos 213 mutantes não eram detectados: 44 sobreviveram e 12 ficaram sem cobertura. A diferença vem do fato de que cobertura mede apenas se uma linha foi **executada**, e não se o teste **verifica** o resultado dela. Os testes usavam um único valor "feliz" por função, nunca testavam o caminho `false`, não testavam fronteiras (0, valores iguais) e usavam `toThrow()` sem verificar a mensagem.

2. Análise de mutantes críticos

a) isPrimo: laço removido (linha 74, BlockStatement)



O Stryker trocou o corpo do `for` por `{}`. Sem o laço, a função devolve `true` para qualquer número maior que 1. O único teste era `isPrimo(7)`, e 7 é primo, então o resultado continuava certo. A suíte nunca verificava um número composto.

b) medianaArray: ordenação removida (linha 109, MethodExpression)



O Stryker removeu o `.sort(...)`. O teste usava `[1, 2, 3, 4, 5]`, que **já está ordenado**, então ordenar ou não dava no mesmo. Além disso, nenhum teste usava array de tamanho par, e o ramo da linha 112 nem era executado.

c) celsiusParaFahrenheit: operador trocado (linha 93, ArithmeticOperator)



O Stryker trocou `celsius * 9/5` por `celsius * 9 * 5`. O teste usava `celsiusParaFahrenheit(0)`, e zero multiplicado por qualquer coisa continua zero: original e mutante devolvem 32. O valor de entrada “anulava” o operador.

3. Solução implementada

Foram criados 47 testes em `test/operacoes.mutacao.test.js`, e o teste de divisão por zero passou a verificar a mensagem do erro. Os testes que matam os três mutantes acima:

```
// a) números compostos e a fronteira n = 1
expect(isPrimo(4)).toBe(false);
expect(isPrimo(9)).toBe(false);
expect(isPrimo(1)).toBe(false);

// b) entrada desordenada e array de tamanho par
expect(medianaArray([5, 1, 3])).toBe(3);
expect(medianaArray([4, 1, 3, 2])).toBe(2.5);

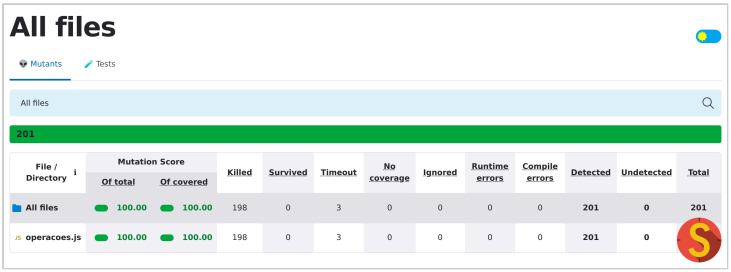
// c) valores diferentes de zero
expect(celsiusParaFahrenheit(100)).toBe(212);
expect(fahrenheitParaCelsius(212)).toBe(100);
```

Por que são eficazes: cada teste usa uma entrada em que o código original e o mutado produzem resultados diferentes, e compara com o valor exato (`toBe`). Os números compostos forçam o `return false` dentro do laço. A entrada desordenada faz o elemento do meio ser diferente da mediana se o array não for ordenado. O valor 100 faz `* 9/5` (212) e `* 9 * 5` (4532) divergirem.

A mesma ideia foi aplicada aos demais sobreviventes: testes com resultado `false` (`isPar(7)`, `isEqual(7, 8)`), valores iguais (`isMaiorQue(5, 5)`), o zero como fronteira (`raizQuadrada(0)`, `fatorial(0)`) e mensagens de erro exatas em `toThrow('...')`.

4. Resultados finais

	Antes	**Depois**
Testes	50	98
Cobertura (stmts / branch / funcs / lines)	85 / 59 / 100 / 99%	100 / 100 / 100 / 100%
Mutantes sobreviventes + sem cobertura	56	0
Mutation score	**73,71%**	**100%**



Mutantes equivalentes: depois dos novos testes restaram 5 sobreviventes (score de 97,65%) que não podem ser mortos, porque o código mutado se comporta igual ao original:

- linha 19: `if (n === 0 || n === 1) return 1` em `fatorial` (4 mutantes). Sem esse `if`, o laço não executa para 0 ou 1 e o resultado também é 1.
- linha 84: `if (numeros.length === 0) return 1` em `produtoArray` (1 mutante). `[].reduce(fn, 1)` já devolve 1.

A equivalência foi confirmada comparando o original com os mutantes em 386 entradas diferentes, sem nenhuma divergência. Como o código em `src/` não podia ser alterado, essas duas linhas foram excluídas da análise no `stryker.conf.json`. Assim, o score final de 100% considera os 201 mutantes restantes.

5. Conclusão

O trabalho mostrou na prática que cobertura alta não significa testes bons: com quase 99% das linhas executadas, um quarto dos bugs introduzidos passava despercebido. O teste de mutação mede o que realmente importa, que é se os testes falham quando o código está errado. Ele também aponta exatamente onde está a fraqueza (fronteiras, caminhos não verificados, valores que escondem erros). Além disso, revelou código redundante no projeto: os mutantes equivalentes indicam verificações que poderiam ser removidas.